

HDOC-30: Menu-Driven Array Operations in C

Bubble Sort & Binary Search | Insertion & Deletion | Right Rotation

Question

Write a C program to perform the following menu-driven operations on a given array:

1. Sort the elements using Bubble Sort algorithm and perform a Binary search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit

Aim

To implement a menu-driven C program that performs sorting and searching, insertion and deletion, and right rotation on an array.

Algorithm

1. Input the size of the array and its elements.
2. Display a menu repeatedly until the user selects Exit.
3. For option 1, apply Bubble Sort, display the sorted array, input a key, and perform Binary Search.
4. For option 2, ask whether to insert or delete. For insertion, input a valid position and value, shift elements right, and insert. For deletion, input a valid position, shift later elements left, and reduce the size.
5. For option 3, input k, reduce it using $k \% n$, and rotate the array to the right k times.
6. For option 4, terminate the program.
7. For any other choice, display an invalid-choice message.

C Program

```
#include <stdio.h>

void display(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
```

```

        if (a[j] > a[j + 1]) {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void) {
    int a[100], n, choice, i, key, result;
    int subChoice, pos, value, k, last;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    do {
        printf("\n1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Rotate array right by k positions\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                bubbleSort(a, n);
                printf("Sorted array: ");
                display(a, n);

                printf("Enter element to search: ");
                scanf("%d", &key);
                result = binarySearch(a, n, key);

```

```

if (result == -1)
    printf("Element not found.\n");
else
    printf("Element found at position %d.\n", result + 1);
break;

case 2:
    printf("1. Insertion\n2. Deletion\nEnter choice: ");
    scanf("%d", &subChoice);

    if (subChoice == 1) {
        if (n >= 100) {
            printf("Array is full.\n");
            break;
        }
        printf("Enter position (1 to %d): ", n + 1);
        scanf("%d", &pos);
        if (pos < 1 || pos > n + 1) {
            printf("Invalid position.\n");
            break;
        }
        printf("Enter value: ");
        scanf("%d", &value);
        for (i = n; i >= pos; i--)
            a[i] = a[i - 1];
        a[pos - 1] = value;
        n++;
        printf("Array after insertion: ");
        display(a, n);
    } else if (subChoice == 2) {
        printf("Enter position (1 to %d): ", n);
        scanf("%d", &pos);
        if (pos < 1 || pos > n) {
            printf("Invalid position.\n");
            break;
        }
        for (i = pos - 1; i < n - 1; i++)
            a[i] = a[i + 1];
        n--;
        printf("Array after deletion: ");
        display(a, n);
    } else {
        printf("Invalid choice.\n");
    }
    break;

case 3:
    if (n == 0) {
        printf("Array is empty.\n");
        break;
    }
    printf("Enter k: ");

```

```
scanf("%d", &k);
k = k % n;
while (k > 0) {
    last = a[n - 1];
    for (i = n - 1; i > 0; i--)
        a[i] = a[i - 1];
    a[0] = last;
    k--;
}
printf("Array after right rotation: ");
display(a, n);
break;

case 4:
    printf("Program terminated.\n");
    break;

default:
    printf("Invalid choice.\n");
}
} while (choice != 4);

return 0;
}
```

Test / Output Screenshots

The following terminal screenshots show successful compilation and execution of all required menu-driven operations.

Screenshot	Operation to show	Suggested input / action
1	Compilation + initial array	gcc array.c -o array then ./array; n = 6; elements = 8 3 12 5 1 9
2	Bubble Sort + successful Binary Search	Choose 1; search key = 9
3	Insertion	Choose 2 -> 1; position = 3; value = 7
4	Deletion	Choose 2 -> 2; position = 4
5	Right rotation	Choose 3; k = 2
6	Exit	Choose 4 and show Program terminated

Expected Result

The menu-driven C program was implemented to perform Bubble Sort with Binary Search, insertion and deletion at valid positions, right rotation by k positions, and exit successfully.

Actual Output / Execution Screenshots

```
(kali㉿kali)-[~/Desktop/apple]
$ gcc hdoc30.c -o hdoc30

(kali㉿kali)-[~/Desktop/apple]
$
```

Figure 1: Compilation of the C program

```
(kali㉿kali)-[~/Desktop/apple]
$ ./hdoc30
Enter number of elements: 6
Enter 6 elements: 8 3 12 5 1 9

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
```

Figure 2: Program execution, initial array input, and menu

```
Enter your choice: 1
Sorted array: 1 3 5 8 9 12
Enter element to search: 9
Element found at position 5.
```

Figure 3: Bubble Sort and Binary Search output

```
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 2
1. Insertion
2. Deletion
Enter choice: 1
Enter position (1 to 7): 3
Enter value: 7
Array after insertion: 1 3 7 5 8 9 12
```

Figure 4: Insertion at position 3

```
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 2
1. Insertion
2. Deletion
Enter choice: 2
Enter position (1 to 7): 4
Array after deletion: 1 3 7 8 9 12
```

Figure 5: Deletion from position 4

```
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 3
Enter k: 2
Array after right rotation: 9 12 1 3 7 8
```

Figure 6: Right rotation by 2 positions

```
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array right by k positions
4. Exit
Enter your choice: 4
Program terminated.
```

```
(kali@kali)-[~/Desktop/apple]
$ ./hdoc30
```

Figure 7: Exit and program termination

Final Result

The menu-driven C program was compiled and executed successfully. Bubble Sort with Binary Search, insertion, deletion, right rotation by k positions, and Exit were tested successfully with the required outputs.